# CodeMentor AI — How It Works

Static Analysis + Local Compilers + Dual-Pass LLM Architecture (Layer Report K0 - K6)

7-LAYER PIPELINE | DETERMINISTIC CORE | 0% SOLUTION LEAKAGE | 33 ADR STANDARDS

## 1. Architectural Pipeline




## 2. Layer-by-Layer Engineering

### Layer 0: Input & Normalization (K0)

Incoming source code undergoes newline (LF/CRLF) and character encoding (UTF-8) normalization. Programming language and game engine context (Unity / Unreal) are identified automatically. Inputs larger than 50 KB are rejected to safeguard resources.

### Layer 1: Pre-Validation & Early Exit (K1)

**Tier 0 (tree-sitter):** AST parse tree evaluates `ERROR` and `MISSING` nodes. Syntax mistakes trigger an immediate early exit with localized diagnostics. **Zero LLM tokens are consumed (~1 ms response).**

**Tier 1 (Local Compilers):** If gcc, clang, or dotnet/Roslyn are detected, the code is evaluated via `-fsyntax-only`. The code is **never executed**.

### Layer 2: Deterministic Static Analysis (K2)

Tree-sitter AST queries gather factual data: loop nesting depth, function call graph, and allocations. 41 catalog rules detect known bugs (memory leaks, division by zero, null dereference). Big-O time and space bounds are deterministically computed.

### Layer 3: Secure Prompt Assembly (K3)

User code is treated strictly as untrusted input. Encapsulation inside `<user_code>` blocks neutralizes prompt injection attempts. Supplying static AST facts directly inside the prompt prevents LLM hallucinations, boosting Big-O accuracy from 85% to 96%.

### Layers 4 & 5: Dual-Pass LLM & Leakage Guard (K4 - K5)

- **Pass A (Deep Analysis):** Evaluates logic flaws, edge cases, and runtime complexity. Validated against JSON schema and strict line-citation proofs.
- **Pass B (Socratic Mentoring):** Translates technical findings into guided questions.
- **Leakage Guard:** Filters questions through regex and semantic barriers. Questions revealing the solution or corrected syntax are rejected and regenerated (0% leakage).

### Layer 6: Multi-Format Reporting (K6)

Synthesizes analysis into a single strongly-typed model rendered across JSON, Markdown, SARIF 2.1.0, and rich desktop / web views.

## 3. Progressive Hinting Architecture

| Hint Level | Pedagogical Scope | Mentor Approach |
|---|---|---|
| **Level 1: Conceptual** | High-level concept | "When heap memory is allocated dynamically, what determines its end of life?" |

| | | |
|---|---|---|
| **Level 2: Line Context** | Specific lines & variables | "After the loop finishes at line 8, has the pointer `tmp` been released?" |
| **Level 3: Directional** | Concrete direction (no code) | "Consider the standard C deallocation function before reaching the return statement." |